

CURSO TÉCNICO SUPERIOR PROFISSIONAL DE PROGRAMAÇÃO DE SISTEMAS DE INFORMAÇÃO
DESENVOLVIMENTO DE APLICAÇÕES

FRAMEWORK E POO

PROGRAMAÇÃO ORIENTADA A OBJETOS

- Baseado na manipulação de **Objectos** que podem conter:
 - Campos com informação normalmente chamados de **Atributos**
(Ex: Altura, Peso, Idade, etc..)
 - Código na forma de procedimentos normalmente chamados **Métodos**
(Ex: Crescer, Engordar, Envelhecer, etc...)
- Existe o molde chamado **Classe** do qual se criam “cópias” chamadas **Instâncias**
 - Classe de um objeto significa o tipo de objeto (Ex: Pessoa)
 - Instância de um objeto significa a realização da classe do objeto (Ex: O João)
- Diferenças para o paradigma procedimental
 - Programação procedimental permite criar um programa passo a passo que segue uma sequência de instruções.
 - Programação procedimental assenta na ideia de que todos os algoritmos são executados com funções e dados que o programador tem acesso e é capaz de mudar.
 - Programação orientada a objetos é mais próxima da forma como funciona o mundo real. Pode dizer-se que comparável ao funcionamento do cérebro humano. Os dados estão protegidos nos objetos.

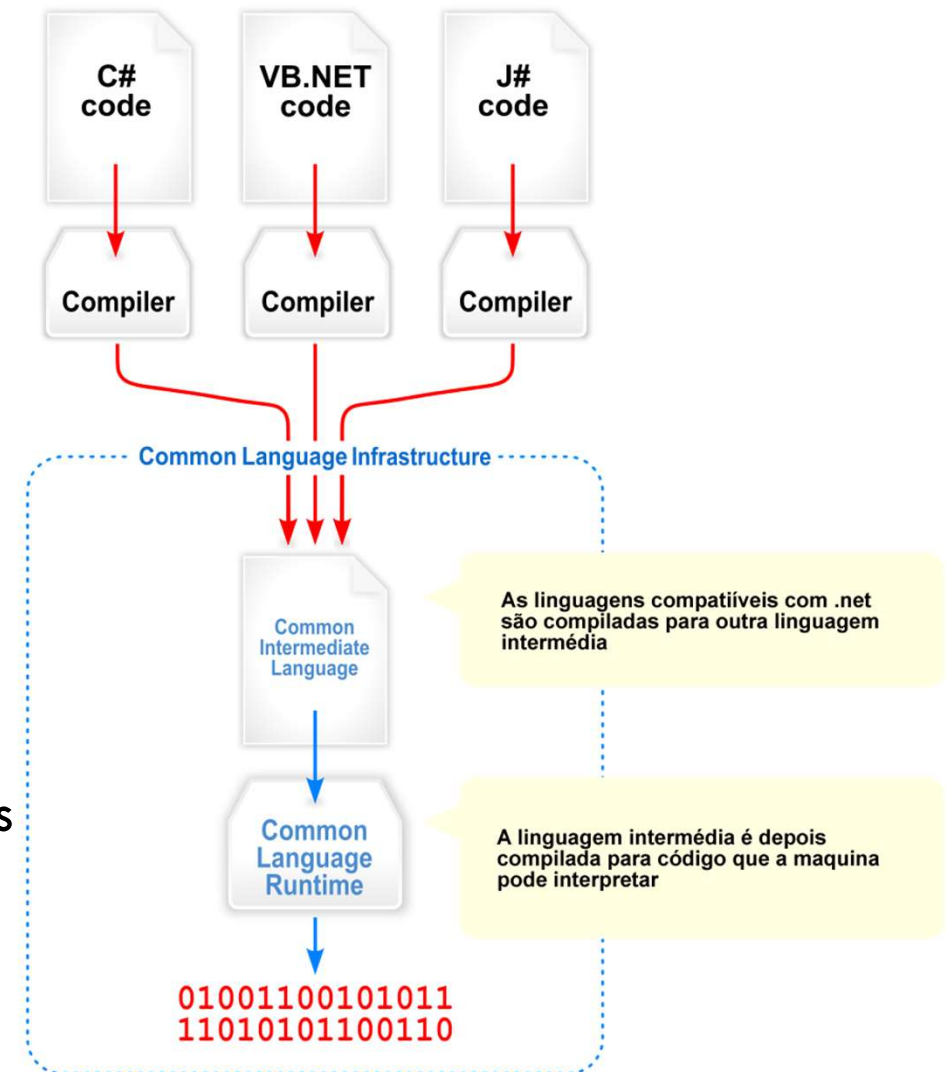
O QUE É UMA FRAMEWORK?

- Conjunto de software que fornece funcionalidades genéricas
- Facilita o desenvolvimento de aplicações (programação)
 - Menos código!
- Código reutilizável
- Constituídos normalmente por:
 - Livrarias de código
 - Programas e Ferramentas de auxílio
 - Visual Studio Debugger
 - Entity Framework, LINQ, ADO.NET
 - Windows Installer
 - Unit Testing
 - Nuget
 - Etc...
 - Compiladores
 - Etc...

Não é necessário reinventar a roda!

FRAMEWORK .NET

- Plataforma de software que liga:
 - Informações (bases de dados, ficheiros, etc)
 - Sistemas (APIs, Serviços Web, etc)
 - Pessoas (Diferentes UIs)
 - Dispositivos (PCs, Móveis, Sensores, etc)
- Propriedade da Microsoft
 - Tem código fechado e aberto
- Pode-se escrever código em diferentes linguagens de programação



C# .NET

C Sharp dot net

- Linguagem de programação muito semelhante ao C e C++ em termos de syntax
- Strongly typed (obriga à definição do tipo de variável)
- Multi-paradigma - imperativa, declarativa, funcional, genérica, orientada a objetos e orientada a componentes
- Tudo é um objeto: *System.Object* é a base de todo o sistema de tipos de C#
- As aplicações C# são executadas num ambiente gerido
 - A gestão de memória é feita em *runtime* pelo GC (*Garbage Collector*)

SINTAX C#

- Using (utilizando)

```
using System;  
using System.Collections;
```

- Namespaces

```
namespace FooNamespace  
{ // Membros  
}
```

- Class

```
class Foo {  
    public Foo(Object value)  
    {  
        // Construtor  
    }  
}
```

SINTAX C#

- If

```
if (i == 3)
{ ... }

else if (i == 2)
{ ... }

else
{ ... }
```

- Switch

```
switch (ch)
{
    case 'A': ...
    break;
    case 'B':
    break;
    case 'C':
    // A switch section can have multiple case labels.
    case 'D': ... break;
    default: ... break;
}
```

SINTAX C#

- While

```
while (i == true)
{
    ...
}
```

- For

```
for (int i = 0; i < 10; i++)
{
    ...
}
```

- Foreach

```
foreach (int i in intList)
{
    ...
}
```


SINTAX C#

- Arrays

```
int[] numbers = new int[2];  
numbers[0] = 2;  
numbers[1] = 5;  
int x = numbers[0];
```

- Com Inicializadores

```
int[] numbers = new int[5]{ 20, 1, 42, 15, 34 };  
int[] numbers2 = { 20, 1, 42, 15, 34 };
```

```
int[,] numbers3 = new int[3, 3] { {2, 3, 2}, {1, 2, 6}, {2, 4, 5} };
```